

MCP servers versus ad-hoc tools

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN Session 10 of 12 · English notes

Ad-hoc tools live inside one AI app: you write `get_current_date_time`, register the schema, ship that notebook. **Model Context Protocol (MCP)** is the same idea moved to a *server* other clients can share. Vijay’s line: write the methods once, consume them from Claude Code, Cowork, GitHub Copilot, or your own Messages client.

The N×N integration problem

Picture three internal sources (a database, a document repo, a set of APIs) and three AI apps that all need them. Point-to-point wiring is nine integrations. Change a column in the database and you patch three apps. MCP inserts two roles so apps stop importing vendor SDKs directly:

- **MCP server** — connects to the data source and *exposes tools* (and related resources) over the protocol.
- **MCP client** — lives in the AI application, lists those tools, and lets Claude choose among them like any other tool schema.

When the database contract changes, you update the server. Clients keep the same connection configuration. Isolation is the point, not a new kind of LLM.

OPEN PROTOCOL

MCP is not “Claude-only magic.” It is an open protocol for how applications provide context to models. Confirm current client/server docs at [MCP on docs.claude.com](#) — marketplace UIs in the recording will have moved.

Where the client comes from

In Claude Code / Anthropic agent stacks, the trainer treated the **MCP client as built in**. You do not implement JSON-RPC by hand. You *configure* which servers to attach. The Anthropic SDK likewise ships client helpers. Your job as a platform team is usually to **build or host servers** for proprietary data.

Three server shapes in class

Shape	Who hosts data	Classroom example
Local proprietary server	You, on the same machine as the client	stdio / IO when client and server share a host
Your server wrapping third-party HTTP	You call a public or internal API, then re-expose MCP tools	Demo: get todos, get energy prices
Vendor-provided server	Figma, Google, Playwright, ElevenLabs, ...	Figma MCP in Claude Cowork; Gmail connector also MCP

Cowork “connectors” in this course are MCP (or MCP-shaped) attachments. Default *code* tools you wrote in the notebook are not; they are in-process functions. Third-party SaaS is where MCP is the default pattern today.

Marketplaces and registries

You do not have to invent every server. The session listed MCP marketplaces and registries (VS Code marketplace, GitHub MCP registry, Claude’s own server catalogue). Playwright’s listing, for example, advertised tools such as browser size, close, and console messages — an LLM then chooses among them.

- Prefer the **vendor’s own** MCP server when you consume Figma (or similar), not a random third-party clone.
- If *you* publish a server, any library that implements the protocol is valid (FastMCP, official SDKs, FastAPI wrappers). Anthropic’s libraries know the edge cases; protocol completeness matters more than brand.
- A **registry** is closer to an npm/marketplace listing than to OpenAPI Swagger. MCP is not “just REST with extra steps”: discovery, tool list, and connect artefacts are protocol-specific. You might bundle a Docker image (server you run locally) or a “bundle as service” (remote URL plus connect config).

Transports: IO, socket, HTTP

The demo FastAPI server exposed four tools: add two numbers, get energy prices, get todos, greeting. The same tool list can be reached over different channels:

- **IO / stdio** — client and server on one machine (typical local Claude Code MCP).
- **Socket** — same network, remote process; the trainer compared it to a WebSocket-style wrapper but noted sockets can ride other application protocols (SMTP, FTP) if you are bridging those systems.
- **HTTP** — server on the public internet; you still do not hand-write every body field the way a one-off REST client would. The MCP client fills the protocol envelope.

GitHub Copilot’s `mcp.json` in that demo was “where is my server URL (or command)?” The Copilot MCP client then listed tools. After a window reload and with the process actually running, `add / energy prices / todos` appeared. Same server, multiple consumers.

Ad-hoc tool versus MCP tool — exam contrast

	In-process tool	MCP tool
Defined where	Your AI app source	An MCP server process
Reuse	Copy/paste or a shared library	Any MCP client that can reach the server
Change management	Redeploy each app	Redeploy the server; clients keep config
Claude’s view	A schema in the tools array	The same kind of schema, discovered via the client

MCP does not replace RAG or memory. It is a connector pattern so context (tools, and often resources) can be provided in a standard way. Session 10 then pivoted to RAG as the next context technique — continued in module 11.

Official documentation

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Client/server model for Claude products and the API.

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

The handshake MCP tools still participate in.

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Where many engineers first attach MCP servers.

- **Cowork / Claude desktop support**

<https://support.claude.com/>

Connectors (Figma, Gmail) as used in earlier Cowork sessions.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.